OPEN AGENT HACKATHON 2026 · TRACK 02 · THE AGENT THAT CAN EXPLAIN WHY

x402 Adoption Investigator

A multi-agent system that answers "how real is x402 adoption?" and proves every number it says: each claim links to a URL with the hash of the bytes fetched, a Base transaction, or a line of its own log.

Team Automata · Martin Pisacreta (solo, Argentina) · Meterless · Zetaris · Cursor

THE USER AND THE PROBLEM

One host, five sources, five numbers

User: the operator of an autonomous agent that sells or buys over x402 and must decide where the real demand is. Today every source gives a different answer, and none of them says why.

SOURCE	WHAT IT MEASURES	ENDPOINTS FOR TIENDA.TEXTOKIT.STORE
CDP Bazaar search	listed resources, capped at 20 results	**17**
Store openapi.json	declared paid operations	**17**
Agent402	paid tools (of 34 tools)	**17**
vet402	resolved endpoints	**19**
x402-trust	keyword search matches	**36**

Activity disagrees too: Bazaar 30-day counters sum to 60 calls, the seller's receipts list 77 settled sales, and 56 of them were paid by the seller itself.

WHY THIS NEEDS AGENTS

A script would pick one number. This system explains all of them.

Live, conflicting sources

Bazaar, the seller's receipts, Base RPC, x402-trust, Agent402 and vet402 each measure a different thing on a different window.

On-chain verification

Settlements are checked on Base by decoding the USDC Transfer log, never trusting tx.from, which is the facilitator.

Disagreement as output

When sources differ, the answer carries a Conflict object with each value, its measurement basis and why they differ.

Failure is normal

Sources go down mid-run. They become explicit "unavailable" entries with a logged recovery, never silent zeros.

WHAT GOES IN, WHAT COMES OUT

A question in, a proof-carrying answer out

Input: one JSON question

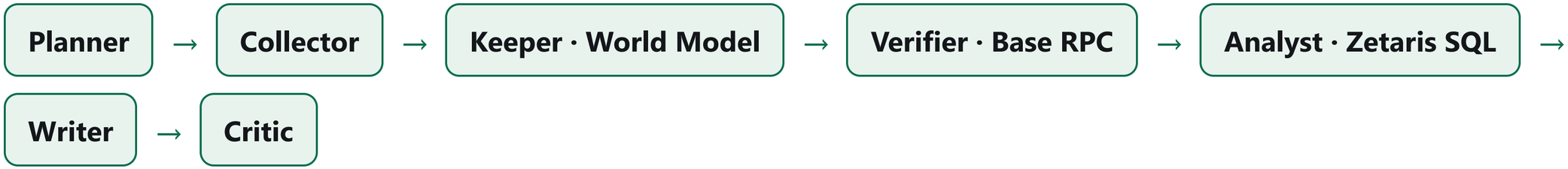
```
{"run_id": "ex01-seller-audit",
 "input": {"question": "How real is the adoption of
   tienda.textokit.store? Who pays, how much,
   do they come back, and why do registries
   disagree about it?"}}
```

Output: one JSON envelope and a Markdown report

- **verdict** chosen by rules, not by the model
- **claims[]** each with evidence: URL + sha256, Base tx, or log line
- **conflicts[]** with per-source values and the explanation
- **rejected_claims[]** with the critique trail
- **unavailable[]** instead of zeros

ARCHITECTURE

Seven roles, bounded loops, every handoff logged



Computed twice

Every metric runs as committed SQL and as a pure Python reducer over the same exports. A claim survives only if both agree.

Critic rules C1 to C7

Evidence, recomputation, scope, staleness, conflict citation, memory caveats, plus a narrative pass that maps every number in the text to an accepted claim.

One event log

handoff, fetch, sql, claim, conflict, critique, retry and recovery lines in logs/<run_id>/run.jsonl.

The generator is used only by Planner, Writer and the Critic's second opinion; the rest is deterministic tooling

A REAL RUN, LINE BY LINE

What a judge sees in run.jsonl

```
guard     network disabled in sample mode (socket.connect patched)
handoff   planner → collector   plan (10 items, sha256 abce43a7…)
handoff   collector → keeper    observations (9, sha256 debfb2…)
recovery  collector  fixture_miss on a Base receipt: settlement marked unverifiable
handoff   verifier → keeper     chain_facts (1)
handoff   keeper → analyst      query_ids (13)
conflict  k01 endpoint_count_for_host   five sources, five measurement bases
conflict  k02 calls_30d_for_host        Bazaar 30-day counter vs seller receipts
handoff   writer → critic       answer
critique  round 1  C1…C7 accept 19 claims · narrative pass: 62 numbers mapped, 0 unsourced
handoff   keeper → hmem         claims (19) · memory feedback written
```

Example 1 emits 108 events: 11 handoffs, 17 fetches, 13 SQL queries, 8 critiques, 5 recoveries, 2 conflicts.

METERLESS, STRUCTURAL

World Model as shared state, H-MEM as policy

World Model

- Entities: sellers, endpoints, payers, facilitators, settlements
- Provenance on every row; facts without a URL, tx or log line are refused by the engine
- Idempotent ingest, proved in every output: the second pass appends 0 events

H-MEM

- Tiered memory store with a trust ledger and feedback per run
- Pitfall memories ("never sum unique payers across endpoints", "receipts are self-reported") force caveats the Critic checks
- Memory ablation committed: removing one memory changes fifteen citations, not the verdict

ZETARIS, THE FEDERATION LAYER

Every number is computed in Lightning SQL

- Receipts, Bazaar catalog and registry snapshots registered as Zetaris sources through the local REST API
- 19 committed analytics queries: sales by wallet class, repeat payers, concentration, registry counts, settlements vs receipts
- The same ANSI SQL runs on SQLite offline; a test asserts both backends return equal rows
- **--require-zetaris** turns any fallback into an error for the live demo

Logged per query

Executor, SQL sha256, row count, time used and tables with source URLs, in sql.jsonl and zetaris.jsonl. Live traces are committed separately from the offline examples.

RESULTS ON REAL DATA

Three questions, three proved answers

Seller audit

Verdict **self_settled_mostly**. 77 sales, 56 paid by the seller itself; 21 external sales by 4 payers, 0.044 USDC in total.

Catalog concentration

Verdict **concentrated**. 28,462 Bazaar resources, 2,680 hosts, 1,785 payTo wallets; the top host holds 29.5% of 824,796 calls in 30 days.

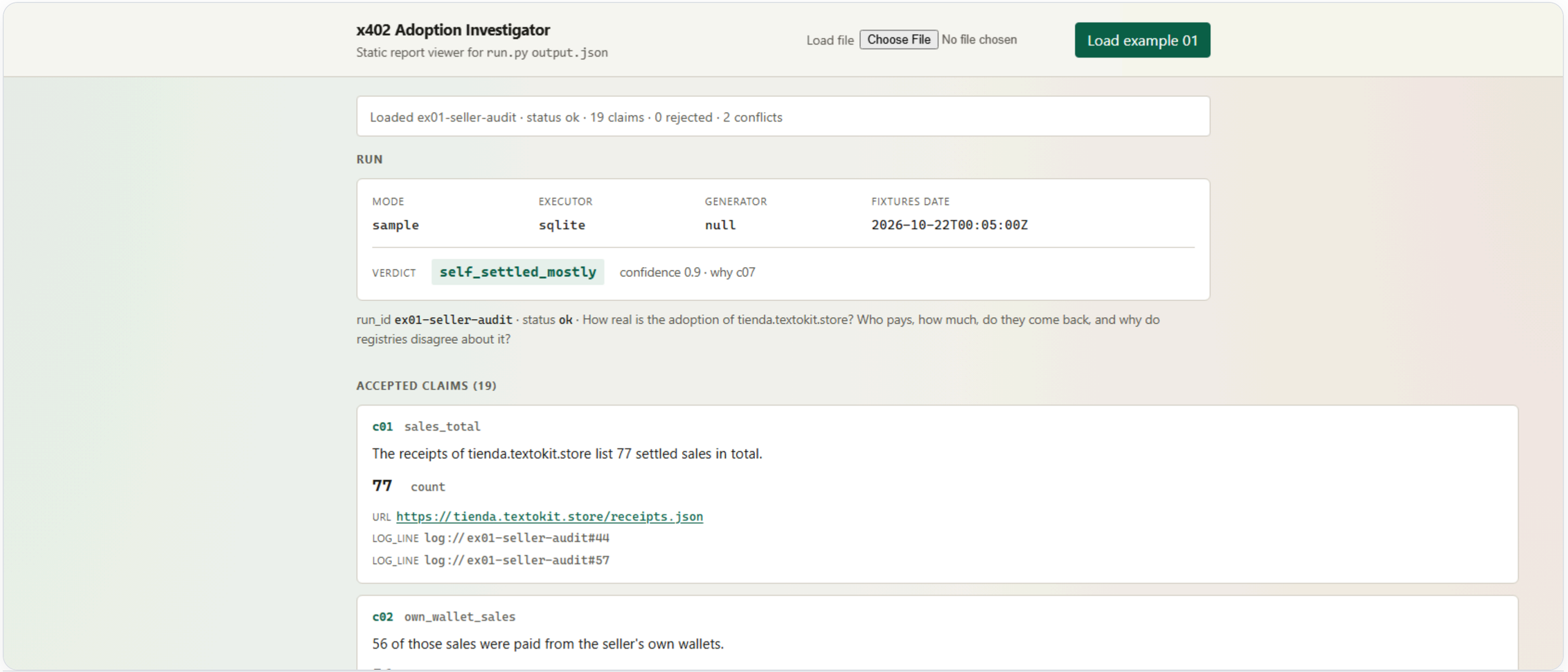
Payer dossier

Verdict **adoption_thin**. One external payer, one purchase of 0.001 USDC, the settlement verified on Base against the receipts.

The honest finding for our own store: most of its sales were self-tests. The system says so, with the proof.

SEE IT

The report viewer



STACK AND HOW TO TRY IT

One command, offline, from a clean clone

```
git clone https://github.com/martinPisacreta/x402-adoption-investigator
cd x402-adoption-investigator
python run.py --sample --input examples/ex1.json
```

Python 3.11, standard library **Meterless H-MEM + World Model** **Zetaris 2.4.3 Lightning SQL** **Cursor rules + Cursor Agent** **Base RPC** **Docker** **193 tests**

Pre-existing component declared: the Automata x402 store (tienda.textokit.store), the data subject of example 1. Everything else was written in the build window.